

1/10 11:07:53 ***



GenAI

[Generative AI]

Good old data mining ['GODaM' :)]...

So far, we have studied many data mining algorithms (including NNs, ie. ML), which all learn (model) patterns (between outputs and inputs) in existing data, and use that to classify/calculate new outputs based on new inputs.

Given existing data: existing inputs → existing outputs

Learn (model) the pattern in the data

Use the model to compute: NEW input → ? NEW output

So... what's this 'GenAI' thing?

'Generative AI' - a revolution [started with GANs]

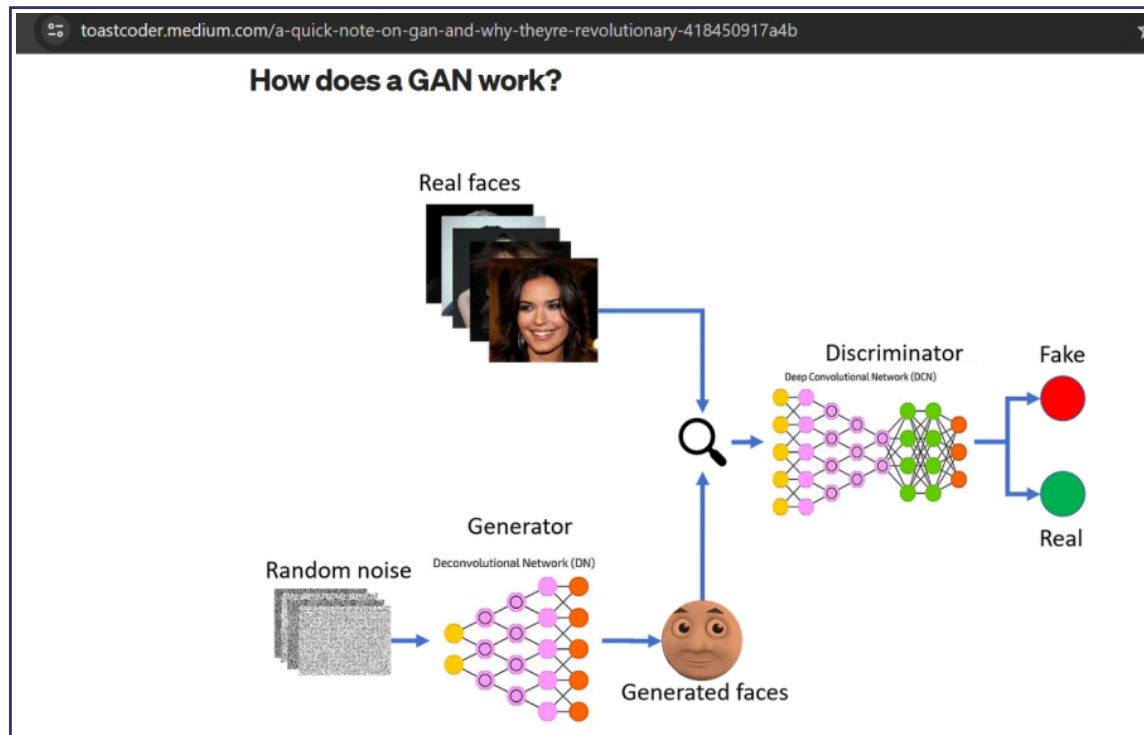
Generative AI, very loosely speaking, 'runs a neural network BACKWARDS'!

Rather than learn to classify new data using existing data, why not GENERATE new data instead?

Researchers tried this, but with unimpressive results.

In 2014, Ian Goodfellow got a much better idea than the 'SOTA' - why not pair up TWO NNs in opposing order - one a generator (eager 'student'), and the other, a discriminator (strict 'teacher')? His invention is called a 'GAN'.

GAN stands for Generative Adversarial Network



More: https://developers.google.com/machine-learning/gan/gan_structure and <https://machinelearningmastery.com/what-are-generative-adversarial-networks-gans/>

An early use of GenAI was/is to "add style" to imagery: <https://arxiv.org/abs/1508.06576>

anmol19005.medium.com/evolution-of-style-transfer-techniques-1a892e796475

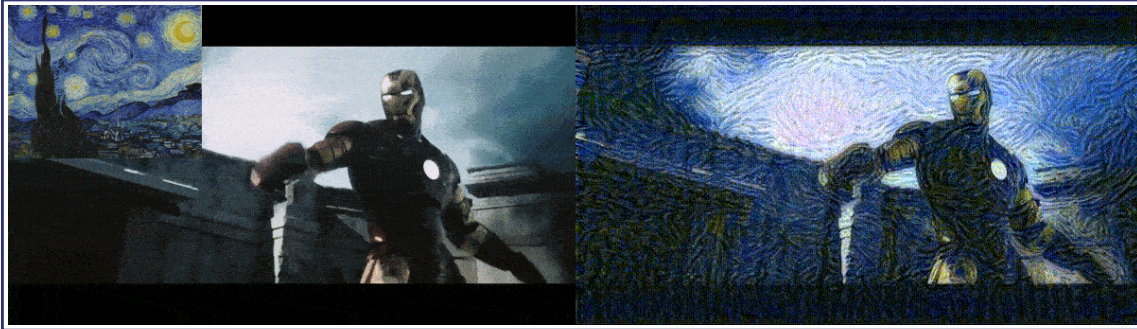
Content ImageStyle ImageOutput Image

The Deep Learning magic

Our starting point was the original algorithm for neural style transfer by [Gatys et al \(2015\)](#) based on optimizing an image to match the content and style of another image by manipulating some clever losses.

In summary, the general idea is:

- Take an image for style (`style_img`)
- Take an image for content (`content_img`)
- Take a pre-trained neural net
- Create an “output” image where each pixel is a parameter to optimize
- Optimize the “output” image so that the content loss and style loss are minimized

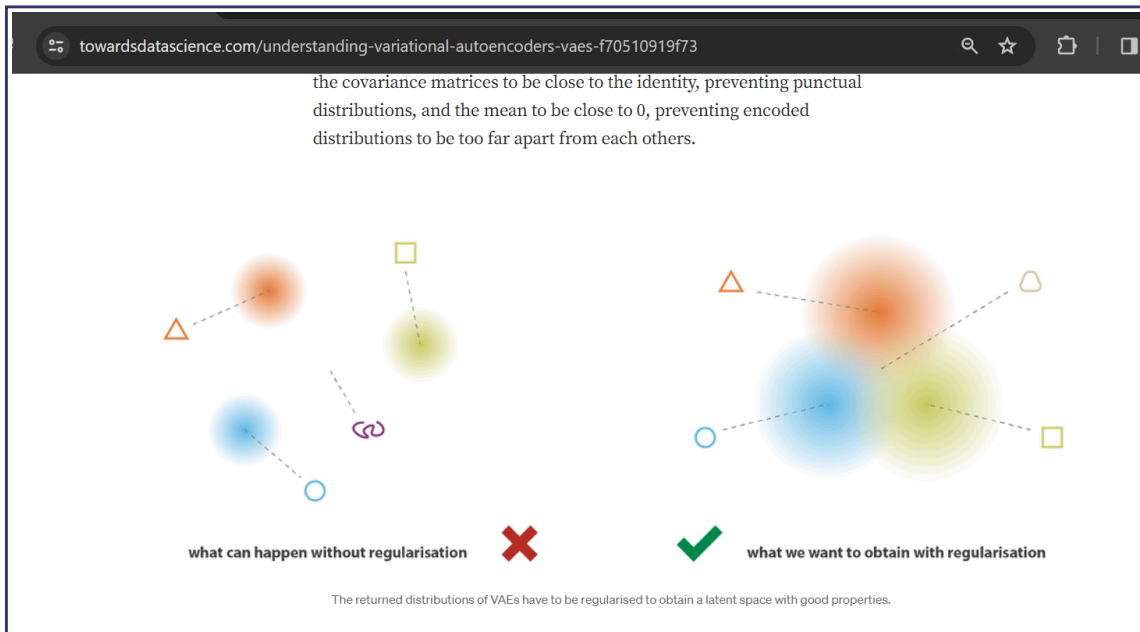


Encoders/decoders, autoencoders, VAEs

Encoder: a function (NN) that maps original input to LATENT/ENCODED space [decoder is the reverse]

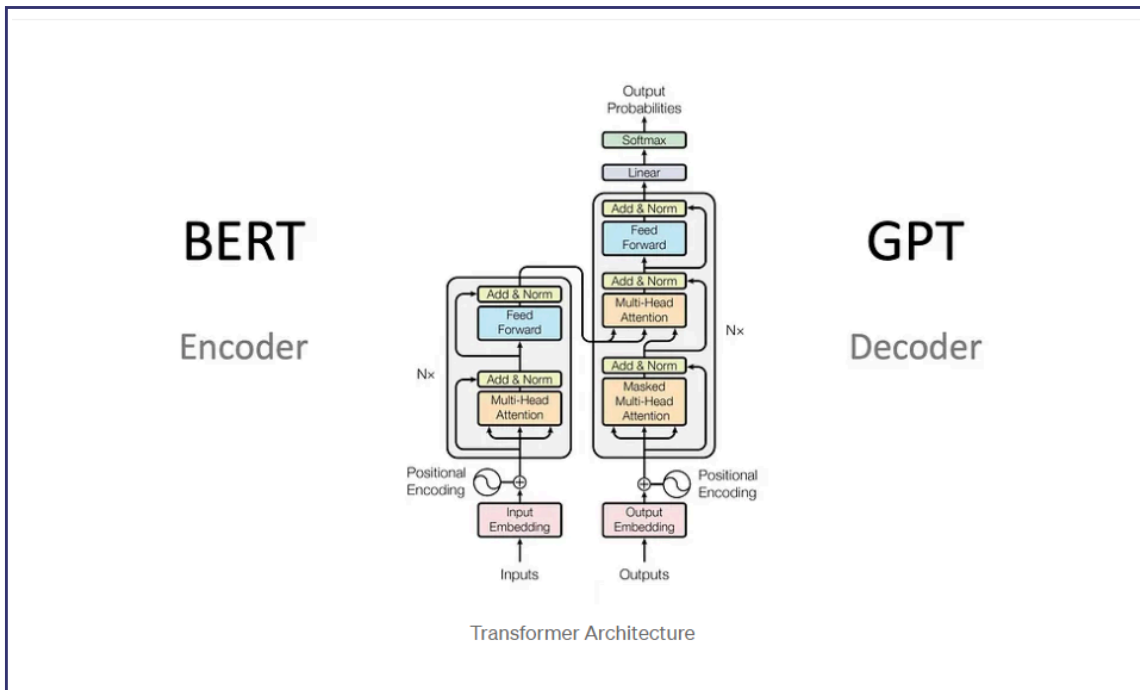
Autoencoder: encoder + decoder combination - can GENERATE NEW OUTPUT (by interpolating a random point in latent space)!

Variational AE: the encoder produces a distribution rather than a single point [and the decoder uses a sampled point from the distribution].



Transformers

'Attention is all you need': <https://arxiv.org/abs/1706.03762>



Revolution: non-fixed size and parallelizable self-attention mechanism (ie. computing word affinities).

More:

- <https://jalammar.github.io/illustrated-transformer/>
- <https://medium.com/@yulemoon/detailed-explanations-of-transformer-step-by-step-dc32d90b3a98>
- https://www.youtube.com/watch?v=zjkBMFhNj_g
- Stefania: <https://machinelearningmastery.com/the-transformer-attention-mechanism/>
- Vinija: https://vinija.ai/models/LLM/#google_vignette
- Self vs cross attention

The decoder takes a prompt (new point in latent space), INTERPOLATES over inputs (ALL English!!), generates output.

Transformers can be adapted to other domains, eg. ViT [<https://www.linkedin.com/pulse/vision-transformer-damien-benveniste-phd-nygoc>], time-series prediction, music gen, etc.

It all comes down to TRAINING a transformer, using DATA... Here are 3 ways...

Context

Because 'context is everything', we need to address the 'quadratic bottleneck' of attention computation. Here are ways:

Indeed, in DNA models like EVO, the researchers used the Hyena operator instead of attention to avoid the previously mentioned quadratic relationship. Hyena operators use **long convolutions** instead of attention to capture long-range dependencies with a subquadratic cost.

| *and cheaper.*

Other alternatives aim for a hybrid approach that, instead of ditching attention altogether, finds the sweet spot between attention and other operators to maintain performance with lower costs.

Recent examples include Jamba, which cleverly mixes Transformers with other, more efficient architectures like *Mamba*.

| *Mamba, Hyena, Attention... you are probably thinking I'm just name-dropping fancy words to prove a point.*

Signup Here

Read in Browser

AlphaSignal

Hey ,

Welcome to today's edition of AlphaSignal.

Whether you are a researcher, engineer, developer, or data scientist, our summaries ensure you're always up-to-date with the latest breakthroughs in AI.

Let's get into it!

Lior

IN TODAY'S SIGNAL

- **Top Paper:** Google New Infinite Context Method
- **Trending Repos:**
 - gemini-cookbook
 - aider
 - ragflow
- **Top Lecture:** Pretraining an LLM on Unlabeled Data

Read Time: 4 min 06 sec

Enjoying this newsletter? Please forward it to a friend or colleague. It will help us keep this content free.

TOP NEWS

Language Models

Google Releases New Infinite Context Method

👤 1683 🗨️ 394

Rouge overall score

17

18

19

20

16K

32K

64K

128K

256K

500K

18.02

18.83

18.71

19.43

19.23

19.34

Input length

What's New

Can LLMs Handle Unlimited Context? Google researchers introduced a new concept called Infini-attention in their latest paper, enabling LLMs to process inputs of any length.

Comparison with Traditional Transformers:

Typical transformers reset their attention memory after each context window to manage new data, losing previous context. For example, in a 500K token document split into 100K token windows, each segment starts fresh without memory from the others.

Infini-attention's Approach:

Infini-attention retains and compresses the attention memory from all previous segments. This means in the same 500K document, each 100K window maintains access to the full document's context.

The model compresses and reuses key-value states across all segments, allowing it to pull relevant information from any part of the document.

How Infini-attention Works:

- Utilizes standard local attention mechanisms found in transformers.
- Integrates a global attention mechanism through a compression technique.
- Merges both local and global attention to manage extended contexts efficiently.

In other words, the method effectively gives each window a view of the entire document, achieving what's termed as "infinite context."

Key Performance Metrics:

- **1B Model:** Effectively manages sequences up to 1 million tokens.
- **8B Model:** Achieves state-of-the-art results in tasks like summarizing books up to 500K tokens in length.

Key Highlights:

- **Memory Efficiency:** Constant memory footprint regardless of sequence length.
- **Computational Efficiency:** Reduces computational overhead compared to standard mechanisms.
- **Scalability:** Adapts to very long sequences without the need for retraining from scratch.

Why This Matters

Elvis Saravia: "Given how important long-context LLMs are becoming having an effective memory system could unlock powerful reasoning, planning, continual adaption, and capabilities not seen before in LLMs. Great paper!"

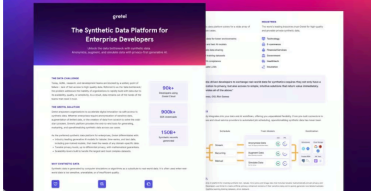
Community Feedback

swyx: "with Griffin and Infini-attention, it increasingly feels like Google leapfrogged Together and RWKV in the race for scaling up linear attention, and they shared a watercooler conversation with Anthropic or something"

Raúl Avilés Poblador: "Sounds good on paper but scaling this to near-infinity would also mean inference would require crazy hardware resources, isn't it?"

READ THE PAPER

Get the PyTorch Implementation



Revolutionize AI Development with Gretel's Synthetic Data Solutions

Facing data privacy roadblocks in your AI projects? Our partner Gretel's synthetic data solutions create realistic, anonymized datasets that enable robust AI training without risking sensitive information.

Gretel's groundbreaking synthetic data platform speeds up development cycles and helps you meet stringent privacy standards with ease, enabling your team to innovate safely and efficiently.

We highly recommend you download their one-page solution brief to learn more!

DOWNLOAD BRIEF

partner with us

TOP OF GITHUB

Language Models

gemini-cookbook

A collection of guides and examples for the Gemini API, including quickstart tutorials for writing prompts and using different features of the API, and examples of things you can build.

★ 1573

Code Assistants

aider

Aider is a command line tool that lets you pair program with GPT-3.5/GPT-4, to edit code stored in your local git repository. Aider will directly edit the code in your local source files, and git commit the changes with sensible commit messages. You can start a new project or work with an existing git repo. Aider is unique in that it lets you ask for changes to pre-existing, larger codebases.

★ 8922

RAG

ragflow

RAGFlow is an open-source RAG (Retrieval-Augmented Generation) engine based on deep document understanding. It offers a streamlined RAG workflow for businesses of any scale, combining LLM (Large Language Models) to provide truthful question-answering capabilities, backed by well-founded citations from various complex formatted data.

★ 4709

Give Your Honest Opinions on Building with AI

How satisfied are you with your current AI stack? Do you think your company is allocating GPUs correctly? What are you most excited to see in the next generation of models and tools?

Share your thoughts on the evolving state of AI in the newest State of AI survey. (You could win a \$500 Amazon gift card!)

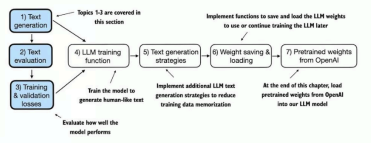
Share thoughts

TOP LECTURE

Language Models

Build an LLM from Scratch Chapter 5: "Pretraining an LLM on Unlabeled Data"

★ 13,532



What's New

Chapter 5 of Sebastian Raschka's "Build an LLM from Scratch" book, titled "Pretraining an LLM on Unlabeled Data," is now available. This chapter advances the series by focusing on the implementation of a training function and the initiation of pretraining for the LLM.

Key topics covered include:

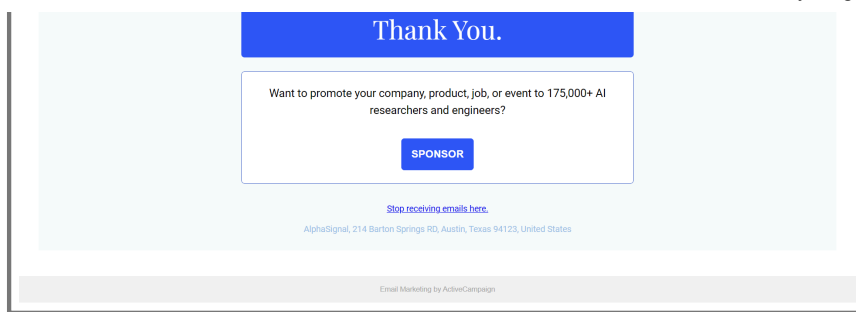
- Computing the training and validation set losses to assess the quality of text generated by the LLM during training.
- Implementing a training function and starting the pretraining process.
- Techniques for saving and loading model weights, allowing for the continuation of training at different stages.
- Loading pretrained weights from OpenAI to enhance model performance.

CHECK THE REPO

How was today's email?

Not Great Good Amazing

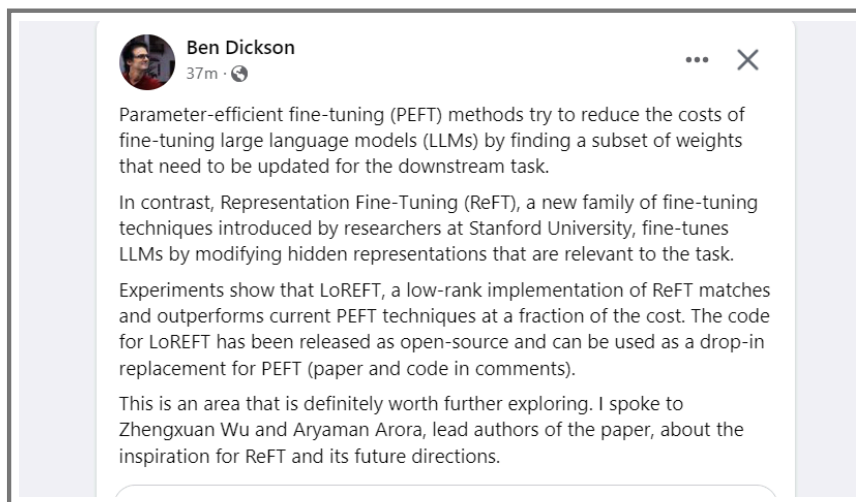
Shita is an editor at AlphaSignal and AI Engineer at Clear Ventures. Experienced in Computer Vision and Language models, she holds 2 masters in Applied Mathematics.



PFFFFFF-FT!!

A way to augment an LLM's general learning, is to 'fine tune' (FT) it for specialized domains such as medicine, law, etc.

ReFT is the latest FT technique:



RAGGGG-GG

Another way to augment a query's context is to use external memory - this can be KG, DBs, text...

But there can be 'issues' - context mismatch, incorrectly chunked input...

Speeding up RAG: <https://techcrunch.com/2024/06/21/openai-buys-rockset-to-bolster-its-enterprise-ai/> and <https://openai.com/index/openai-acquires-rockset/> [also [this](#)]

Learning more (WAY more):

- lightning.ai
- deeplearning.ai

GenAI: other (non-plaintext) content

Today we have a variety of content we can generate, using a variety of underlying approaches - generation via diffusion, generation via radiance modeling, generation via point-cloud modeling, generation via musical pattern modeling...

Almost ANYTHING can be generated - molecular representations, architectural plans, circuit layout, business cards, A/V/I/3D, etc etc. :

- CODE!!! eg. [this!](#)
- images, eg. Stable Diffusion, Midjourney...
- video - Sora, Kling, Dream Machine
- music - eg. <https://google-research.github.io/seanet/musiclm/examples/>
- 3D CG - eg. <https://dreamfusion3d.github.io/>, NeRF, 3DGS...
- combos, eg. <https://arcade.studio/>, <https://domoai.app>
- ...

Here is a HUGE list of GenAI-assisted coding platforms!

```

* https://famous.ai
* https://expo.dev/go - app gen! iOS, Android]
* https://base44.com - app gen
* Ollama run :) Ollama+LiteLLM+Streamlit = magic!
* ReplIt
* Bolt.new: https://bolt.new/
* https://cursor.com/en
* https://lovable.dev/ [and https://github.com/mendableai/open-lovable - TO SCRAPE AND CLONE AN ENTIRE SITE!!]
* https://windsurf.com/editor
* Warp: https://www.warp.dev
* https://famous.ai/ [https://go.famous.ai/order-634008011752726719355]
* Claude Desktop, Claude Code CLI [https://www.anthropic.com/claude-code]
* Gemini CLI
* GitHub Copilot
* OpenAI
* huggingface.co/spaces
* Firebase Studio
* https://opal.withgoogle.com/landing/
* https://jules.google/
* simstudio, voltagent
* TransformerLab
* Cursor
* VSCode: via Cline+Continue.dev+Ollama
* https://roocode.com/
* https://www.youtube.com/watch?v=xM4HsMODReQ LiteLLM w/ Ollama
* https://www.youtube.com/watch?v=nQCOTzS5oU0 OpenWebUI
* Bolt.DIY
* LlamaCoder: https://llamacoder.together.ai/ [https://medium.com/augmented-startups/llama-coder-ai-code-generator-llama-3-1-405b-4cb09948748d]
* together.ai, http://llamacoder.together.ai
* 'try Kimi'
* https://www.supadev.so/
* https://www.launch.today/ - Bangalore-based!
* Cline.bot
* Ollama UI, [vs LM Studio, MSTY, and OpenWebUI] https://www.youtube.com/watch?v=prwWESXl7wg
* FOUR for UI/app gen: ffloat, https://uxpilot.ai/Stitch [https://codingbeautydev.com/blog/new-google-stitch]
* https://pre.dev
* https://github.com/mozilla-ai/any-llm - wow... [https://www.theunwindai.com/p/july-2-4]
* https://app.emergent.sh/
* https://www.dronahq.com/veda-ai/
* https://qwenlm.github.io/blog/qwen3-coder and Qwen Code
* https://ellmer.tidyverse.org/ - LLMs from inside 'R'
* https://rork.com/
* https://www.createanything.com/
* https://pypi.org/project/llmswap/ [a lot like any-llm above]

```

Content creation:

```

* https://www.meshy.ai/ Prompts -> models
* https://studio.tripo3d.ai/workspace/generate
* https://hyper3d.ai/
* https://www.assethub.io/new-home - img to 3D etc
* https://www.keyshot.com/keyshot-studio-ai
* https://www.seeles.ai/home - EELE [text -> endless 3D game worlds]
* https://www.worldlabs.ai/
* https://monaverse.ai/ - models!
* https://deepmind.google/discover/blog/genie-3-a-new-frontier-for-world-models/

```

New directions

As is to be expected, the wild new world of LLMs began with 'ChatGPT' in Nov'22 - but is expanding in multiple, very interesting directions!

- new architectures - eg. Mamba, [this](#) ['MLGRU'], KAN...
- hardware
- MCC [eg. NV's 'NIM': <https://nvidianews.nvidia.com/news/generative-ai-microservices-for-developers>, <https://www.nvidia.com/en-us/ai/?videogallery=mg0kwpmUhPU>]
- FT alts, eg. PO
- RAG2.0, no-RAG, Task-Aware RAG...
- SLMs [including Llama, Mixtral, Gemini, Phi-3('mini' version has just 3.8B params!)...] (also, read [this](#))
- MoE
- bit-crushed weights [eg. [this](#)] [q: why does 1.58 figure in this?!]; also, [this](#) illustrates the idea
- multimodal LLMs
- WORLD models, eg. <https://www.worldlabs.ai/blog/marble-world-model>, <https://panworld.ai/>...
- 'embodied' LLMs [eg. <https://www.youtube.com/watch?v=GiKvPJSOUmE>]
- custom-purpose enterprise LLMs, eg. Arctic, Nemotron
- instrumentation: <https://www.rungalileo.io/blog/announcing-rag-and-agent-analytics>
- synthetic data gen: <https://syntagen.github.io/> and real datasets: <https://encord.com/multimodal-dataset-emm1/>
- diffusion models
- reasoning models
- alternative position encoding
- standards: DSPy, MCP (eg. [here](#) is a description, and a [visual](#)), A2A, ADK...
- alternative attention mechanisms, eg. MLA [Multi-head Latent Attention]+DSA [DeepSeek Sparse Attention]: https://www.youtube.com/watch?v=kAEPS_AUGy8 [and older ones such as Hyena, Mamba, Jamba etc]
- next-gen LLMs: <https://research.google/blog/introducing-nested-learning-a-new-ml-paradigm-for-continual-learning/>
- AGENTS!!!
 - <https://www.kaggle.com/whitepaper-introduction-to-agents>
 - <https://www.kaggle.com/whitepaper-agent-tools-and-interoperability-with-mcp>
 - <https://www.kaggle.com/whitepaper-context-engineering-sessions-and-memory>
 - <https://www.kaggle.com/whitepaper-agent-quality>
 - <https://www.kaggle.com/whitepaper-prototype-to-production>
- agentic alternative to fine-tuning (!): <https://arxiv.org/pdf/2510.04618>
- ...

[Here](#) is a compact glossary of terms...